

ПРАКТИЧЕСКАЯ РАБОТА № 5. ОРГАНИЗАЦИЯ ПАМЯТИ.

РАБОТА С ПАМЯТЬЮ

Цель данной практической работы — ознакомиться со основами организации работы с памятью при выполнении программ на языке Си в операционной среде ОС Линукс/Юникс

1.1. Указатели и их использование

Указатель — это специальная переменная, которая содержит адрес памяти другой переменной.

Хранение переменных переменных в памяти

Основная память компьютера называется оперативной памятью (ОЗУ) и используется для хранения переменных и другой информации, связанной с программой.

Оперативная память компьютера состоит из байтов, которые расположены последовательно. Каждый байт имеет порядковый номер, который называется его адресом (адресом памяти).

Если мы создаем какую-либо переменную, компилятору Си необходимо переместить память для переменной в основную память (ОЗУ).

В языке программирования Си мы обычно указываем имя переменной вместе с типом данных переменной во время создания переменной. Чтобы компилятор знал, сколько памяти (сколько байт) он должен выделить для указанной переменной. Давайте рассмотрим программу для получения адреса переменной на языке Си.

```
1  /*
2     Программа демонстрирует работу с адресами в Си
3  */
4
5  #include<stdio.h>
6
7  int main()
8  {
9
10     // declare a variable
11     int score = 300;
12
13     // variable value
14     printf("Value of score: %d\n", score);
15
16     // get the address of 'score' variable
17     printf("Address of score: %p\n", &score);
18 }
```

```
19     return 0;
20 }
21
```

Результат работы программы:

```
1 $ gcc memory-address.c
2 $ ./a.out
3 Value of score: 300
4 Address of score: 0x7fff0034a9f4
5 $
```

Переменная Score хранится в ОЗУ по адресу 0x7fff0034a9f4 .

Объявление переменных-указателей на языке Си

Мы можем объявить переменную-указатель, используя * (символ звездочки). Вначале указывается тип переменной указателя

Синтаксис объявления указателя:

тип_данных *имя_указателя;

Пример объявления переменных указателей

```
// Integer pointer
int * intPtr;
```

```
// character pointer
char * chPtr;
```

```
// we can declare multiple pointers like below
int * p1, *p2, *p3;
```

Мы создали переменную-указатель с именем intPtr , которая является целочисленным указателем. intPtr указывает на целочисленную переменную .

Аналогично, chPtr является указателем символа и указывает на символьные данные. Поэтому нам нужно создать указатель с тем же типом данных, что и данные, на которые он будет указывать.

Размер переменных указателей в языке Си

Переменные указателя также являются переменными. Таким образом, указатели также будут занимать некоторое место в памяти.

Но в отличие от других обычных переменных, все указатели (целочисленные указатели, указатели символов, указатели с плавающей запятой и т. д.) хранят адрес памяти другой переменной.

Поскольку переменная-указатель хранит только адрес памяти, размер переменной-указателя составляет 4 байта (размер указателя будет меняться в зависимости от вашего компилятора и архитектуры ОС).

Вообще же размер указателя любого типа в языке Си равен размеру целочисленной переменной в этой системе. Например, в 16-битной системе размер целого числа составляет 2 байта, что соответствует размеру указателя.

Указатель хранит адрес переменной в памяти, а адрес переменной — это не что иное, как целочисленное значение. Таким образом, переменная-указатель хранит целочисленное значение, что означает, что размер переменной-указателя такой же, как размер целочисленной переменной. Размер переменной-указателя не зависит от типа данных переменной, на которую она указывает, поскольку в конце она указывает на ячейку памяти. Следовательно, размер указателя `char`, указателя `int`, указателя с плавающей запятой, указателя `NULL` и т. д. одинаковы.

Отличие между неинициализированным (диким) указателем и нулевым указателем.

Указатель в Си, который не был инициализирован до первого использования, называется неинициализированным указателем.

Неинициализированный указатель указывает на некоторую случайную ячейку памяти. Указатель `NULL` в языке Си — это указатель, который не указывает ни на что или на базовый адрес сегмента.

Инициализация переменной указателя

По умолчанию переменные указателя будут иметь мусорные значения. Поэтому всегда рекомендуется инициализировать переменные указателя.

Вот синтаксис для инициализации переменных указателя

тип `*имя_указателя = &имя_переменной;`

Пример программы с использованием указателей

```
1      /* Программа демонстрирует работу с указателями в Си
2      */
3
4      #include<stdio.h>
5
6      int main()
7      {
8
9          // объявляем переменную и инициализируем ее
10         int score = 300;
11
12         // объявление и инициализация переменной указателя 'intPtr'
13         // адресом 'score'
14         int *intPtr = &score;
15
16         // variable value
17         printf("Value of score: %d\n", score);
18
19         // получить адрес переменной 'score'
20         printf("Адрес score: %p\n", &score);
21
```

```

22      // значение переменной указателя 'intPtr'
23      printf("Значение intPtr(указателя) :%p\n", intPtr);
24
25      // Адрес переменной указателя 'intPtr'
26      printf("Адрес intPtr(указателя): %p\n", &intPtr);
27
28      return 0;
29  }
30

```

Результат работы программы

```

1      $ gcc memory-address.c
2      $ ./a.out
3      Значение score: 300
4      Адрес score: 0x7ffc08f4a00c
5      Значение intPtr(указателя) :0x7ffc08f4a00c
6      Адрес intPtr(указателя): 0x7ffc08f4a010
7      $

```

Объяснение работы программы

- Мы запустили программу, создав целочисленную переменную с именем `score` и инициализировав ее значением 300 .
- Затем мы создали целочисленный указатель с именем `intPtr` и инициализировали его адресом целочисленной переменной `score` .
 - Здесь следует отметить: мы использовали звездочку (`*`) для создания переменной-указателя. (т.е. `int * intPtr`)
 - Мы также использовали адрес оператора (`&`), чтобы получить адрес целочисленной переменной оценки (т. е. `&score`) .
- Выводим значение переменной `score`, используя `printf` («Значение score: `%d\n` », `score`) ; печатает 300
- Затем мы отобрали адрес переменной `score` , используя операцию получения адреса `&` . Она печатает адрес переменной `score` , который в данном случае равен `0x7ffc08f4a00c` .
 - **`printf("Адрес оценки: %p\n", &score);`**
- Затем мы напечатали значение переменной-указателя `intPtr`, используя другую функцию `printf` . Здесь мы использовали спецификатор формата `%p` , поскольку `intPtr` содержит адрес оценки . Если вы заметили, значение `intPtr` и адрес переменной оценки совпадают. то есть : `0x7ffc08f4a00c`
 - `printf ( "Значение intPtr(указатель):%p\n" , intPtr ) ;`
- Наконец, переменная-указатель `intPtr` также будет иметь адрес в памяти. Чтобы напечатать адрес переменной -указателя `intPtr` , мы использовали **`&intPtr`** . что равно `0x7ffc08f4a010`

- `printf("Адрес intPtr(указатель): %p\n", &intPtr);`

Что следует помнить при использовании указателей в Си

1) Обязательно присвойте адрес памяти переменной переменной-указателю.

```
1 // правильно
2 int n1 = 44;
3 int *p1 = &n1;
```

Приведенный выше пример — правильный способ присвоить адрес переменной в памяти переменной-указателю.

```
1 // неправильно
2 int n1 = 44;
3 int p2 = &n2;
```

Если вы заметили, мы пропустили символ звездочки (*) перед p2. Таким образом, переменная p2 не является переменной-указателем. Это просто еще одна целочисленная переменная, поэтому переменная p2 не может хранить адрес другой переменной.

2) Если вы объявляете и назначаете переменные указателя, обязательно используйте указатель при назначении адреса.

Мы объявили указатель ptr3, а затем присвоили ptr3 адрес переменной n3. Теперь ptr3 хранит адрес памяти n3.

```
1 // неверный способ
2 int n4 = 63;
3 int *ptr4;
4 *ptr4 = &n4; // Not correct
```

Мы использовали * ptr4 для обновления адреса &n4. Это неправильно. Мы не используем оператор разыменования (*) для обновления адресов.

Мы должны использовать оператор разыменования (*) только для обновления значения переменной, на которую указывает указатель.

В приведенном выше случае переменная n4 теперь будет иметь значение, равное ее собственному адресу в памяти. (что-то вроде 130295644).

1.1.1 Указатель типа void

Согласно стандарту Си, целочисленное константное выражение со значением 0 или такое выражение, приведённое к типу [void *](#), называется константой нулевого указателя. Если константа нулевого указателя преобразуется в тип указателя, результирующий указатель называется

нулевым указателем.

Синтаксис указателя типа `void` в языке Си:

```
void * vPtr ;
```

Здесь `void` — это тип, а `vPtr` — это указатель `void`.

Указатель типа `void`, это универсальный тип в языке Си. Мы можем назначить адрес любого типа данных указателю `void`. Поэтому давайте присвоим адрес целочисленного значения указателю `void`. Объявим переменную `ed`

```
int ed = 364;
```

Объявим указатель `vPtr` тип `void`

```
void * vPtr;
```

Присвоим адрес целочисленной переменной `ed` указателю `void vPtr` .

```
vPtr = &ed;
```

Теперь указатель `void vPtr` хранит адрес целочисленной переменной `ed` .

Пример программы

```
1  /*
2  Программа демонстрирует работу с void указателем
3
4  */
5  #include<stdio.h>
6
7  int main()
8  {
9      // create an integer variable
10     int ed = 364;
11
12     // declare a void pointer
13     void *vPtr;
14
15     // assign address of integer to void ptr.
16     vPtr = &ed;
17
18     // print the addresses.
19     printf("Address of ed variabel (&ed) : %p\n", &ed);
20     printf("Value of void pointer (vPtr): %p\n", vPtr);
21
22     return 0;
23 }
24
```

В приведенной выше программе мы создали целочисленную переменную с именем `ed` и сохранили значение 364. Затем мы создали указатель `void` с именем `vPtr` . Поскольку указатель `void` может хранить адрес переменной другого типа, мы сохранили адрес края целочисленной переменной в указателе `void vPtr` .

Вывод программы

```
1 $ ./a.out
2 Value of the ed is : 364
3 Value of ed using void pointer : 364
4 $
```

1.2. Ошибки при работе с указателями

1.2.1. Дикие указатели (wild pointer)

Дикий указатель — это переменная-указатель, которая не указывает на действительный адрес памяти. Неинициализированные переменные указателя являются хорошим примером дикого указателя в Си.

Давайте посмотрим на пример дикого указателя на языке Си.

```
1 // объявление указателя
2 int * wPtr;
```

Здесь указатель `wPtr` является диким указателем, поскольку мы не инициализировали его каким-либо адресом.

Доступ к дикому указателю в Си

Доступ к диким указателям приведет к неопределенному поведению. Это может привести к ошибкам в программе, а иногда и к сбою программы.

```
// неопределенное поведение
1 printf("Value pointed by wild pointer is :%d\n", *wPtr);
2 //wPtr это дикий указатель
```

В приведенном выше примере мы попытались получить доступ к значению, указанному `wPtr`. Поскольку `wPtr` является диким указателем, это приведет к неопределенному поведению.

Давайте скомпилируем программу.

```
1 $ gcc wild-pointer.c
```

Мы использовали компилятор `gcc` для компиляции программы, и он генерирует `a.out` как исполняемый файл.

```
1 $ ./a.out
2 Segmentation fault
3 $
```

Как мы видим из приведенного выше вывода, разыменованное дикое указатель `wPtr(*wPtr)` вызвало **ошибку сегментации**.

Ошибка сегментации возникает, когда вы пытаетесь получить доступ к памяти, которая вам не выделена. Итак, здесь дикий указатель `wPtr` указывает

на какое-то место с мусором в памяти. Итак, мы получили ошибку сегментации и программа аварийно завершилась

Указатель в С, который не был инициализирован до первого использования, называется диким указателем. Дикий указатель указывает на некоторую случайную ячейку памяти.

```
int main() {
    int *ptr;
    /* Ptr is a wild pointer, as it is not initialized Yet */
    printf("%d", *ptr);
}
```

Для того чтобы избежать этого нужно инициализировать указатель когда он объявляется адресом какого-либо объекта/переменной или NULL;

```
int main() {
    int val = 5;
    int *ptr = &val; /* Initializing pointer */
    /* Ptr is not a wild pointer, it is pointing to the address of variable val */
    printf("%d", *ptr);
}
```

Как правильно обрабатывать дикий указатель на языке Си?

Важно проверить, содержит ли указатель действительный адрес, прежде чем пытаться разыменовывать указатель (получить значение доступа). Для этого можно инициализировать переменные указателя соответствующими значениями. Например мы можем инициализировать переменные указателя символом NULL вместо простого объявления значения.

```
1 // инициализация указателя
2
3 int *ptr = NULL;
```

И также можно проверьте указатель на значение NULL перед разыменованием переменной указателя.

```
// проверка 'ptr' на NULL.
if(ptr != NULL)
{
    printf("value pointed by ptr:%d\n", *ptr);
}
```

Доступ к значению указателя ptr возможен только в том случае, если оно содержит действительный адрес любой переменной.

Пример программы, реализующей проверку

```
1  /*
2   * Программа демонстрирует дикие указатели
3   */
4  #include<stdio.h>
5
6  int main()
7  {
8      // Initialize the pointer with NULL
9      int *wPtr = NULL;
10
11     if(wPtr != NULL)
12     {
13         // print the value
14         printf("Value pointed by wild pointer is :%d\n",
15 *wPtr); //wPtr is a wild Pointer
16     }
17     else
18     {
19         // show warning.
20         printf("Warning: Pointer is Not Initialized!\n");
21     }
22
23     return 0;
24 }
25
```

Теперь в следующей программе мы инициализировали переменную указателя `wPtr` значением `NULL`. При доступе к дикому указателю `wPtr` или его разыменовании мы собираемся проверить, имеет ли указатель по-прежнему `NULL`-адрес. Если он по-прежнему равен `NULL`, то указатель не содержит допустимого адреса, поэтому мы не можем его разыменовать или использовать.

Результат работы программы

В результате выполнения программы мы видим увидим что, дикий указатель `wPtr` не обновляется и по-прежнему имеет значение `NULL`. Мы просто напечатали предупреждающее сообщение: «Внимание: указатель не инициализирован !», Таким образом, мы можем избежать сбоев программы.

Если переменная-указатель `wPtr` действительно обновлена значением переменной, тогда `wPtr` не будет равна `NULL`, поэтому условие `if` будет выполнено, и пользователь получит правильное значение.

1.2.2. Висячие указатели

Висячий указатель — это указатель, указывающий на область памяти, где объект больше не существует. Висячий указатель возникает, когда мы удаляем или освобождаем объект/переменную без изменения значения входящего указателя, так что указатель по-прежнему указывает на ячейку освобожденной памяти.

Предположим, указатель «ptr» указывает на адрес целочисленной переменной «sum». Через некоторое время переменная sum удаляется, но указатель ptr по-прежнему указывает на ту же ячейку памяти.

Указатель ptr не знает, что сумма переменной была удалена. Теперь указатель ptr стал висячим указателем.

Если мы попытаемся получить доступ к указателю местоположения памяти по ptr, то получим мусорное значение.

Вот пример висячего указателя:

```
/* Динамическое выделение памяти для целочисленной переменной
*/
int *ptr = (int*)malloc(sizeof(int));
/* ptr указывает на местоположение в памяти динамически выделяемой переменной
int. Теперь мы освободим ранее выделенную память, используя free*/
free(ptr);
/* Теперь ptr становится висячим указателем */
```

Всякий раз, когда мы освобождаем/удаляем какой-либо объект, мы должны установить все входящие указатели этого объекта на NULL.

```
free(ptr);
ptr = NULL;
```

1.3. Организация работы с памятью

Управление памятью или выделение памяти — это процесс, с помощью которого компьютерные программы распределяются по физическому или виртуальному пространству памяти. Компьютерная память — это ограниченный ресурс, которым необходимо эффективно управлять.

Выделение памяти может выполняться как до, так и во время выполнения программы. Существует два метода распределения памяти: статическое выделение памяти и динамическое выделение памяти.

Статическое распределение памяти

При этом типе распределения компилятор выделяет фиксированный объем памяти во время компиляции, а операционная система внутренне использует структуру данных, известную как стек, для управления памятью.

Точные требования к памяти должны быть известны заранее, поскольку после выделения памяти ее нельзя изменить.

```
int sunDays; //необходимо инициализировать или присвоить значение во время
выполнения
int temp = 0; // обычная переменная
const double maxTemp = 10.35; // константа, нельзя изменить
```

Динамическое распределение памяти

При этом типе распределения системная память управляется во время выполнения. Динамическое управление памятью в языке программирования Си осуществляется с помощью функций `malloc()`, `calloc()`, `realloc()` и `free()`. Эти четыре функции определены в `<stdlib.h>` заголовочном файле стандартной библиотеки С.

Эти функции используют пространство кучи системной памяти.

Функция `calloc()`

Функция `calloc()` используется для динамического выделения массива блоков памяти указанного типа. Каждый блок имеет значение по умолчанию 0.

Синтаксис функции

```
void* calloc(number_of_blocks, size);
```

Пример

В следующем примере демонстрируется `calloc()` функция:

```
int *ptr;  
  
// Указатель ptr содержит адрес первого байта в выделенной памяти  
ptr = (int*) calloc(10, sizeof(int));
```

Приведенный выше оператор выделяет непрерывное пространство в памяти для 10 элементов, каждый из которых имеет размер `int`.

Функция `malloc()`

Функция `malloc()` используется для выделения блока памяти в куче. Он выделяет пользователю указанное количество байтов, но не инициализирует. После выделения программа обращается к этому блоку памяти через возвращаемый указатель `malloc()`.

Синтаксис функции

```
void* malloc(size);
```

Параметр `size` имеет тип `size_t`. Указатель по умолчанию, возвращаемый функцией, `malloc()` имеет тип `void*`, но его можно преобразовать в указатель любого типа данных. Однако если места недостаточно для объема памяти, запрошенного `malloc()`, выделение завершается неудачей и `NULL` возвращается указатель.

Пример

В следующем примере демонстрируется `malloc()` функция:

```
int *ptr;

// // Указатель ptr содержит адрес первого байта в выделенной памяти
ptr = (int*) malloc(10 * sizeof(int));
```

Поскольку размер `int` составляет 4 байта, приведенный выше оператор выделит 40 байтов ($4 * 10$) памяти.

Функция `free()`

Функция `free()` используется для динамического освобождения памяти во время выполнения. Память, выделенная функциями `malloc()` и `calloc()`, должна быть освобождена вручную, когда она не используется. Освобождение помогает уменьшить потери памяти.

Синтаксис

```
free(memoryblock);
```

Выделенная память `memoryblock` должна существовать заранее, чтобы функция `free()` могла освободить ее.

Пример

В следующем примере демонстрируется использование `free()` функции:

```
int *ptr;

//Память, выделенная с помощью функции malloc()
ptr = (int*) malloc(10 * sizeof(int));

// Память освобождается с помощью функции free()
free(ptr);
```

Функция `realloc()`

Функция `realloc()` используется для перераспределения блока памяти. Если памяти, ранее выделенной с помощью функции `malloc()` или `calloc()`, недостаточно, ее можно перераспределить с помощью этой `realloc()` функции.

Синтаксис функции

```
realloc(memoryblock, size);
```

Память `memoryblock` должна существовать ранее, до использования функции `realloc()`, причем `size` имеет тип `size_t`

Пример

В следующем примере демонстрируется `realloc()` функция:

```
int *ptr;

// Память, выделенная с помощью функции malloc()
ptr = (int*) malloc(5 * sizeof(int));

// ptr перераспределяется с 40 байтами (4 * 10) памяти
ptr = realloc(ptr, 10 * sizeof(int));
```

1.4. Утечки памяти в Си программах

Утечка памяти происходит, когда программист выделил память в куче, но не освободил ее обратно в кучу. Утечка памяти уменьшает доступную память для программы и, как следствие, снижает производительность программы.

Вот пример утечки памяти:

```
#include <stdio .h>
#include <stdlib.h>

void getSomething(){
    /* Dynamically declare memory for an integer array of 10 elements */
    int *array = (int *) malloc(10*sizeof(int));
    /* Do something and return without releasing allocated memory */
    return;
}

int main() {
    int i;
    for(i = 0; i<100; i++){
        getSomething();
    }
    return 0;
}
```

В приведенной выше программе функция `getSomething` динамически выделяет память в виде массива, но затем возвращает ее, не освобождая ее. Каждый раз, когда вызывается функция `getSomething`, она уменьшает доступную память для программы. Чтобы избежать утечек памяти, функция `getSomething` должна освободить выделенную память, используя `free`.

```
void getSomething(){
```

```

/* Dynamically declare memory for an integer array of 10 elements */
int *array = (int *) malloc(10*sizeof(int));
/* Do something and release allocated memory */
free(array);
return;
}

```

Используйте утилиту Valgrind для профилирования и поиска утечек памяти в программах

```
% valgrind --tool=memcheck --leak-check=full ./a.out
```

1.5. Указатели на функции

Константный указатель всегда указывает на одну и ту же ячейку памяти. Как только константный указатель инициализируется для указания на ячейку памяти, мы не можем изменить его, чтобы он указывал на какую-то другую ячейку памяти. Однако мы можем изменить данные, хранящиеся в этом месте памяти.

<https://metanit.com/cpp/tutorial/4.8.php>

<https://en.cppreference.com/w/cpp/language/pointer>

1.6. Порядок выполнения практической работы

1. Прочитайте теоретический материал, посвященный работе с динамической памятью
2. Рассмотрите и проанализируйте программные примеры.
3. Напишите программу, которая динамически размещаем массив из N целых чисел. Проанализируйте адреса.
4. Напишите программу, которая динамически размещаем двумерный массив из N целых чисел. Проанализируйте адреса.
5. Написать программу в соответствии с заданием из файла с приложением (Приложение к практике 5)
6. Отчет должен содержать полную информацию по разделам о проделанной вами работе и выводы
7. Оформите отчет по практической работе, загрузите в свой слот СДО МИРЭА и сдайте преподавателю

Примечание. Отчет должен содержать: титульный лист, название, цель работы и описание выполняемого задания, программы и результат

выполнения программ.

Контрольные вопросы

1. Поясните как создаются процессы в ОС Линукс и Юникс системах.
2. Какой номер у самого первого процесса в системе?
3. Поясните у всех ли процессов в системе есть родительские процессы?
4. Как посмотреть процессы, которые выполняются на вашем терминале?
5. Как посмотреть процессы, которые выполняются в вашей вычислительной системе?
6. Как определить загрузку процессора вашей системе?
7. Как получить сведения о системе с которой вы работаете?
8. В каком состоянии могут находиться запущенные процессы?
9. Как посмотреть информацию о потребляемых процессами системных ресурсах?
10. Что означает понятие приоритет процесса и как он назначается.
11. Какие значения приоритетов могут принимать процессы?
12. Что означает значения PID и PPID?
13. Как посмотреть список задач и их состояний в вычислительной системе?
14. Как посмотреть степень загрузки системы?

Практическая работа №5